SIMATS ENGINEERING

PROGRAMMING IN JAVA
ASSIGNMENT 3 — CO3

Exception Handling and Multithreading

Student Name	D. Saketh
Register Number	192525018
Course	CSA09 — Programming in Java
JDK	OpenJDK 21
GitHub URL	Pending repository creation
Submission mode	Individual preparation; faculty approval required

Pair-work declaration

The supplied Assignment 3 brief requires two students per group and requires both students to execute all questions. No partner details were provided. This report records only D. Saketh and does not claim another student's work. Confirm with faculty that individual submission is permitted.

Replace the GitHub placeholder after creating the required repository.

Q1 — Handling Failure in a Bank ATM Simulation (6 marks)

Objective: Validate withdrawal inputs and classify malformed input, invalid amounts and insufficient funds.

Implementation and reasoning: The program parses the amount, checks positivity and balance, and separates input and domain failures. finally executes for each attempt. Specific exception types should be caught before broader types.

Alternative / complexity analysis: Using Scanner.hasNextInt() and error codes avoids exception construction for routine invalid input but requires explicit checks. Both approaches are O(1) time and auxiliary space per withdrawal; throwing exceptions has higher failure-path overhead.

Observed execution: Five paths tested: 200, abc, -10, 900 and 50. finally executed on every attempt; final balance was 250.

OpenJDK 21 | Q1_ATM

```
$ javac ATMDemo.java
$ java ATMDemo

200 => SUCCESS; balance=300.0
  finally executed
abc => INPUT ERROR: Amount must be a whole number
  finally executed
-10 => BUSINESS ERROR: Amount must be positive
  finally executed
900 => BUSINESS ERROR: Insufficient balance
  finally executed
50 => SUCCESS; balance=250.0
  finally executed
```

Q2 — User-Defined Exceptions and Propagation (7 marks)

Objective: Represent portal storage failures with a checked hierarchy and preserve the original cause.

Implementation and reasoning: ExamException extends Exception; SubmissionStorageException extends ExamException. Storage.save wraps IOException using a cause constructor and the controller reports a safe message while retaining diagnostics.

Alternative / complexity analysis: Unchecked exceptions remove the catch-or-declare requirement but make failure handling less explicit. Stack-trace capture costs time and memory proportional to stack depth; chaining retains the cause and its trace.

Observed execution: Output matched the brief: Saved answer01.pdf; Error: Could not store answer02.pdf; Cause: Disk quota exceeded.

OpenJDK 21 | Q2_Exceptions

```
$ javac StorageDemo.java
$ java StorageDemo

Saved answer01.pdf
Error: Could not store answer02.pdf
Cause: Disk quota exceeded
```

Q3 — Resource Safety, Debugging and Robust Code (7 marks)

Objective: Generate a sales report from multiple files without leaking resources on failure.

Implementation and reasoning: Nested try-with-resources closes each reader and the output writer, including when a later file is missing. The failure path reports NoSuchFileException.

Alternative / complexity analysis: For total input size N across f files, processing is O(N). Sequential processing holds at most one reader plus one writer open: O(1) handles. Manual close logic must cover partial initialization and exceptions during close.

Observed execution: Two valid inputs generated the summary; a missing file was handled as NoSuchFileException.

OpenJDK 21 | Q3_Resources

```
$ javac ResourceDemo.java
$ java ResourceDemo

Store 1: 1200 | Store 2: 850 |
Handled input failure: NoSuchFileException
Try-with-resources closes opened files on success and failure.
```

Q4 — Thread Creation and the Thread Life Cycle (7 marks)

Objective: Run four scanner stations concurrently and wait for all stations.

Implementation and reasoning: Each Station extends Thread, pauses, and reports its count. start() creates concurrent execution; join() ensures the total is printed after all stations finish. Calling run() directly does not create a new thread.

Alternative / complexity analysis: Runnable separates task from thread. ExecutorService reuses a bounded worker pool; idealized work is distributed over about n/p batches. Scheduling order is nondeterministic.

Observed execution: The four stations reported 25, 30, 20 and 25; total 100 printed after completion. Ten-run, priority and daemon experiments remain to be recorded locally.

OpenJDK 21 | Q4_Threads

```
$ javac ThreadLifecycleDemo.java
$ java ThreadLifecycleDemo

Station-4 scanned 25 scripts
Station-3 scanned 20 scripts
Station-2 scanned 30 scripts
Station-1 scanned 25 scripts
All stations completed; total scanned = 100
```

Q5 — Race Conditions in a Cinema Seat-Booking System (6 marks)

Objective: Make check-then-book atomic to prevent selling the same seat twice.

Implementation and reasoning: The synchronized book() method protects checking, setting the seat flag and incrementing the count under one intrinsic lock.

Alternative / complexity analysis: Without synchronization, two threads may both observe a free seat. AtomicBoolean.compareAndSet() is an alternative. Storage is O(s) for s seats; atomic wrappers add per-seat overhead.

Observed execution: 100 threads competed for 10 seats; synchronized version reported exactly 10 sales. Unsynchronized baseline measurements are still required.

```
OpenJDK 21 | Q5_Synchronization

$ javac SeatBooking.java
$ java SeatBooking

Seats sold: 10
```

Q6 — Deadlock and Inter-Thread Communication (7 marks)

Objective: Implement a bounded producer-consumer buffer and explain deadlock prevention.

Implementation and reasoning: TokenCounter uses a circular array. put() waits while full; take() waits while empty; both use synchronized methods and notifyAll(). Wallet locks should be acquired in a stable global order.

Alternative / complexity analysis: wait() releases the monitor; sleep() does not. while handles spurious wakeups and re-checks conditions. ArrayBlockingQueue can replace manual wait/notify; tryLock(timeout) needs retry/rollback policy.

Observed execution: Two consumers collected tokens 1–20; total was 210, as expected. Buffer capacity is five.

```
OpenJDK 21 | Q6_Deadlock_ProducerConsumer

$ javac TokenCounter.java
$ java TokenCounter

Sum of tokens collected: 210 (expected 210)
```

Q7 — Design a Concurrent, Fault-Tolerant Processor (10 marks)

Objective: Process score files concurrently while isolating corrupt/missing inputs and producing a complete summary.

Implementation and reasoning: A fixed ExecutorService runs per-file tasks. Each task returns an Outcome; Future.get() collects results, so one bad file does not stop the run.

Alternative / complexity analysis: For n files and p workers, idealized processing is roughly n/p batches. One thread per file creates many thread stacks; a fixed pool bounds concurrency. Actual runtime depends on CPU and I/O.

Observed execution: With pool size 4, 7 of 9 inputs succeeded and 2 failed (malformed and missing file). Pool-size timings for 1,2,4,8 must be measured locally.

```
OpenJDK 21 | Q7_ConcurrentProcessor

$ javac ResultProcessor.java
$ java ResultProcessor

OK student1.txt score=10
OK student5.txt score=50
FAILED student3.txt NumberFormatException: For input string: "bad"
OK student7.txt score=70
OK student4.txt score=40
OK student2.txt score=20
OK student6.txt score=60
OK student8.txt score=80
FAILED missing.txt NoSuchFileException: /tmp/results10712931884165995986/missing.txt
SUMMARY success=7, failed=2, total=9, pool=4
```

Individual Contribution Record

The brief requests Student A, Student B and Joint responsibilities. Only the known student's work is recorded; no partner attribution is claimed.

Question	Student A — D. Saketh	Student B	Joint
Q1	Prepared / executed	Not assigned	Not claimed
Q2	Prepared / executed	Not assigned	Not claimed
Q3	Prepared / executed	Not assigned	Not claimed
Q4	Prepared / executed	Not assigned	Not claimed
Q5	Prepared / executed	Not assigned	Not claimed
Q6	Prepared / executed	Not assigned	Not claimed
Q7	Prepared / executed	Not assigned	Not claimed

Remaining requirements before submission

• Obtain faculty approval for individual submission under the pair-work rule.

• Create the public/private repository named CSA09-192525018-Assignment3 and replace the GitHub placeholder.

• Record Q4 ten-run finish orders, priority and daemon observations.

• Run Q5 unsynchronized baseline for ten runs and compare timings.

• Measure Q7 using pool sizes 1, 2, 4 and 8; record repeated timings and correct summaries.

• Add any additional evidence required by your faculty.

All seven source programs compiled with javac 21 and executed successfully in the preparation environment. Screenshot images are rendered from the captured stdout of those actual runs. No benchmark dataset or second student's work has been fabricated. Source basis: supplied 11-page Assignment 3 — CO3 question paper.